QP CODE 25900786

MAHATMA GANDHI UNIVERSITY, KOTTAYAM

MGU-UGP (HONOURS) REGULAR/ IMPROVEMENT/ REAPPEARANCE

EXAMINATION NOVEMBER 2025 FIRST SEMESTER

Skill Enhancement Course (SEC) - MG1SECBCA100 - FUNDAMENTALS OF PROGRAMMING USING C

(2024 ADMISSION ONWARDS)

ANSWER KEY

Duration: 2 Hours **Maximum Marks: 70**

Remember(K), Understand(U), Apply(A), Analyse (An), Evaluate(E), Create(C), Skill(S), Interest(I) and Appreciation (Ap)

Students should attempt at least one question from each course outcome to enhance their overall outcome attainability.

Part A
Very Short Answer Questions Answer All Questions
Each Question carries 2 marks

1. Describe the purpose of interpreter. [U] [CO1]

The purpose of an interpreter is to translate and execute source code line-by-line, allowing for immediate execution and debugging, rather than compiling the entire code beforehand.

2. Define tokens and list various tokens. [U] / [CO2]

Tokens are the basic building blocks of a program, representing keywords, identifiers, literals, operators, and symbols.

Various tokens in C include:

1. Keywords (e.g., if, while, int)
2. Identifiers (e.g., variable names)
3. Constants (e.g., integers, characters)
4. Strings
5. Operators (e.g., +, -, , /)
6. Special symbols (e.g., (), {}, ;, ,)

3.Define the goto statement and list two reasons why it is generally avoided in programming. [U] / [CO3]

The goto statement Is a jump statement that transfers control to a labeled statement.

Two reasons it's generally avoided:

1. Goto statements can make code difficult to read and understand, leading to maintenance issues.
2. Goto statements can lead to unexpected program flow, increasing the likelihood of errors and bugs.

4.Implement a function to concatenate two strings. [A] / [CO4]

```
#include <stdio.h>
#include <string.h>
void main() {
   char str1[100] = "hello, ";
   char str2[] = "world!";
```

```
    strcat(str1, str2);
    printf("%s\n", str1);
    getch();
}
```

5.Compare the use of * and & in C pointers. [U] / [CO4]

* (dereference operator):
- Used to access the value at the address held by a pointer.
- Example: `int x = ptr;` (accesses the value at the address ptr)

& (Address-of operator):
- Used to get the memory address of a variable.
- Example: `int ptr = &x;` (stores the address of x in ptr)

6.Explain syntax error in programming with an example. [U] / [CO1]

A syntax error occurs when the code violates the language's grammar rules.

Example:

```
void main() {
    int x = 5
    return 0;
}
```

error: missing semicolon at the end of the statement.

corrected code:

```
int main() {
    int x = 5;
    return 0;
}
```

7.Explain how high-level language simplifies the process of writing program. [U] / [CO1]

High-level languages simplify programming by:
1. Abstraction: Hiding low-level details, allowing focus on logic.
2. Portability: Code can run on multiple platforms with minimal changes.
3. Easy syntax: Using natural language-like syntax, making coding easier.
4. Built-in libraries: Providing pre-written functions for common tasks.
5. Error handling: Reducing memory management and runtime errors.

(Consider other valid features also)

8.What is the difference between & and && in C? [U] / [CO2]

- `&` is a bitwise AND operator, performing binary operations on integers.
- `&&` is a logical AND operator, used for conditional statements, evaluating true or false.

Example:

```
int x = 5;  // binary: 101
int y = 3;  // binary: 011
int result1 = x & y;  // bitwise and: 001 (1 in decimal)
int result2 = (x > 0) && (y > 0);  // logical and: true (1 in decimal)
```

9.Define qualifier? [K] / [CO2]

In programming, a qualifier is a keyword that modifies the behavior or properties of a variable, function, or data type.

Examples in C:
- `const`: Indicates a variable's value cannot be changed.
- `volatile`: Indicates a variable can change unexpectedly (e.g., due to external factors).
- `static`: Affects a variable's scope or storage duration.

10.Why scanf() is considered as a formatted input function? [U] / [CO3]

scanf() is a formatted input function as it reads input according to a specified format string, using format specifiers like %d, %c, %f, etc. to parse the input into the required data types.

[10x2 = 20]

Part B

Short Answer Questions Answer any 5 out of 7 Questions Each Question carries 6 marks

11.How do we check whether a given number is a perfect number or not. [A perfect number is a positive integer that is equal to the sum of its proper divisors, excluding itself. Example: The number 6 is a perfect number because its proper divisors are 1, 2, and 3 and 1+2+3=6] [An] / [CO3]

```
#include <stdio.h>
void main() {
   int num, sum = 0;
   printf("enter a number: ");
   scanf("%d", &num);
   for (int i = 1; i < num; i++) {
      if (num % i == 0)
         sum += i;
   }
   if (sum == num)
      printf("%d is a perfect number.\n", num);
   else
      printf("%d is not a perfect number.\n", num);
   getch():;
}
```

12.Construct a C program that initializes an array to store the marks of five subjects for a student. Write a program that calculates and displays the student's grade based on their average score with the following criteria:
'A' grade for an average of 85 and above, 'B' for 70-84, 'C' for 50-69, 'D' for 35- 49, and 'F' for below 35. Also add error handling to manage cases where input marks are invalid (such as negative numbers or values exceeding 100) [A] / [CO4]

```
#include <stdio.h>
void main() {
   int marks[5];
   float sum = 0, average;
   printf("enter marks for 5 subjects:\n");
   for (int i = 0; i < 5; i++) {
      printf("subject %d: ", i + 1);
      scanf("%d", &marks[i]);
      // input validation
      while (marks[i] < 0 || marks[i] > 100) {
         printf("invalid input! marks should be between 0 and 100. re-enter: ");
         scanf("%d", &marks[i]);
      }
      sum += marks[i];
   }
   average = sum / 5;
   printf("average score: %.2f\n", average);
   char grade;
   if (average >= 85)
      grade = 'a';
```

```
    else if (average >= 70)
        grade = 'b';
    else if (average >= 50)
        grade = 'c';
    else if (average >= 35)
        grade = 'd';
    else
        grade = 'f';
    printf("grade: %c\n", grade);
    getch();
}
```

13.Explain how to calculate the sum of diagonal elements in a two-dimensional array. Write an example program for a 3x3 matrix to compute this.

```
#include <stdio.h>
void main() {
    int matrix[3][3];
    int primary_sum = 0, secondary_sum = 0;
    printf("enter elements for 3x3 matrix:\n");
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            printf("element [%d][%d]: ", i, j);
            scanf("%d", &matrix[i][j]);
        }
    }
    for (int i = 0; i < 3; i++) {
        primary_sum += matrix[i][i];
        secondary_sum += matrix[i][3 – i – 1];
    }
    printf("matrix:\n");
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
    printf("primary diagonal sum: %d\n", primary_sum);
    printf("secondary diagonal sum: %d\n", secondary_sum);
    getch();
}
```

14.List and describe the different types of decision-making statements available in C. Provide the syntax for each type [U] / [CO3]

C provides several decision-making statements:

1. if statement: used for simple conditional execution.

syntax:

```
if (condition) {
    // code to execute
}
```

2. if-else statement: used for binary conditional execution.

syntax:

```
if (condition) {
   // code if true
} else {
   // code if false
}
```
3. if-else if ladder: used for multiple conditional checks.

syntax:
```
if (condition1) {
   // code1
} else if (condition2) {
   // code2
} else {
   // default code
}
```
4. switch statement: used for multi-way decision making based on integer values.

syntax:
```
switch (expression) {
   case value1:
      // code1
      break;
   case value2:
      // code2
      break;
   default:
      // default code
}
```
15.List the precedence and associativity of the following operators: +, , =, &&, ||, ==. Give examples. [K] / [CO2]

1. – (Multiplication) – Precedence: 3, Associativity: Left-to-right

Example: `a b c` -> `(a b) c`

2. – (Addition) – Precedence: 4, Associativity: Left-to-right

Example: `a + b + c` -> `(a + b) + c`

3. == (Equality) – Precedence: 7, Associativity: Left-to-right

Example: `a == b == c` -> `(a == b) == c`

4. && (Logical AND) – Precedence: 11, Associativity: Left-to-right

Example: `a && b && c` -> `(a && b) && c`

5. || (Logical OR) – Precedence: 12, Associativity: Left-to-right

Example: `a || b || c` -> `(a || b) || c`

6. = (Assignment) – Precedence: 14, Associativity: Right-to-left

Example: `a = b = c` -> `a = (b = c)`

16.Identify a decision-making statement in programming. Describe how it is used to control program flow. [U] / [CO1]

if statement is a common decision-making statement.

It's used to execute a block of code if a condition is true. Here's how it controls program flow:

- Evaluates the condition
- If true, executes the code block
- If false, skips the code block and continues with the next statement

Example:
```
if (age >= 18) {
   printf("you are eligible to vote.");
```

}
This statement checks if `age` is 18 or more and prints the message if true.

17.Explain the features of structures and demonstrate how to declare and initialize a structure with an example. [U] / [CO4]

Features of Structures:
1. Collection of variables: Structures allow grouping different data types under a single name.
2. User-defined data type: Structures enable creating custom data types.
3. Memory allocation: Memory is allocated for all members when a structure variable is declared.
4. Access members using dot (.) operator: Members are accessed using the dot operator.

Declaring a Structure:

```
Struct Student {
   int rollno;
   char name[50];
   float marks;
};
```

initializing a structure:

```
struct student s1 = {1, “john doe”, 85.5};
```

or

```
struct student s1;
s1.rollno = 1;
strcpy(s1.name, “john doe”);
s1.marks = 85.5;
```

example:

```
#include <stdio.h>
#include <string.h>

struct student {
   int rollno;
   char name[50];
   float marks;
};
void main() {
   struct student s1 = {1, “john doe”, 85.5};
   printf(“roll no: %d\n”, s1.rollno);
   printf(“name: %s\n”, s1.name);
   printf(“marks: %.2f\n”, s1.marks);
getch(); }
```

[5x6 = 30]

Part C

Essay Questions Answer any 2 out of 3 Questions Each Question carries 10 marks

18.Implement a program in C to find the product of digits of an integer number. You have to read the number from the user. [A]/ [CO3]

```
#include <stdio.h>
void main() {
   int num, digit, product = 1;
   printf(“enter an integer: “);
   scanf(“%d”, &num);
   // handle negative numbers
```

```
    num = (num < 0) ? -num : num;
    while (num != 0) {
        digit = num % 10;
        product = digit;
        num /= 10;
    }
    printf("product of digits: %d\n", product);
    getch();
}
```

19.Explain types of modifiers and their importance in C programming. [U] / [CO2]

Types of modifiers in C:

1. Access Modifiers:
   - `auto`: Default storage class for local variables.
   - `extern`: Declares variables/functions defined elsewhere.
   - `static`: Preserves variable value between function calls.
2. Type Modifiers:
   - `signed`: Indicates a variable can hold positive/negative values.
   - `unsigned`: Indicates a variable can hold only non-negative values.
   - `long`: Increases variable size (e.g., `long int`).
   - `short`: Decreases variable size (e.g., `short int`).
3. Type Qualifiers:
   - `const`: Makes variable read-only.
   - `volatile`: Prevents compiler optimization for variable.

Importance:

- Control variable scope and lifetime.
- Optimize memory usage.
- Improve code readability and maintainability.
- Enable compiler checks for data integrity.

Example:

Static int count = 0; // retains value between function calls

Const float PI = 3.14; // read-only variable

20.How can you pass arrays to functions in C? Explain with an example. [U] / [CO4]

In C, arrays can be passed to functions in two ways:

1. Pass by reference: Pass the array name (base address).
2. Pass by value: Not directly possible; pass individual elements.

Example: Pass by reference

```
#include <stdio.h>
void printarray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
}
int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int size = sizeof(arr) / sizeof(arr[0]);
    printarray(arr, size);
    return 0;
}
```

Here, `arr` is passed to `printArray()` by reference. Note:

- `int arr[]` is equivalent to `int arr`.
- The size is passed separately, as it's not available within the function.

- Changes to `arr` in the function affect the original array.

[2x10 = 20]

Prepared By,

MR. VISHNU MOHANAN

ASSISTANT PROFESSOR

DEPARTMENT OF COMPUTER APPLICATIONS

CHINMAYA COLLEGE OF ARTS, COMMERCE & SCIENCE,

TRIPUNITHURA, PIN: 682305

MOB: 9744799308